

POSTGRESQL VS. MONGODB

Comparação de desempenho de SGBD relacional com SGBD NoSQL

POSTGRESQL

CARACTERÍSTICAS

- Modelo: Relacional, dados organizados em tabelas com esquema rígido
- Armazenamento: cada tabela é um arquivo separado em disco
- Mapeamento: Objeto-Relacional, suporte a tipos customizados, herança de tabelas e extensões
- Integridade: Chaves primárias, estrangeiras e constraints garantem consistência dos dados
- Escalabilidade: Vertical
- Linguagem de consulta: SQL
- Transações: Suporte completo a ACID

BENCHMARK

O que é o TPC-H?

- Benchmark padrão para sistemas de suporte à decisão
- Desenvolvido pelo Transaction Processing Performance Council
- 22 consultas analíticas simulando um ambiente comercial
- Amplamente usado para comparar SGBDs

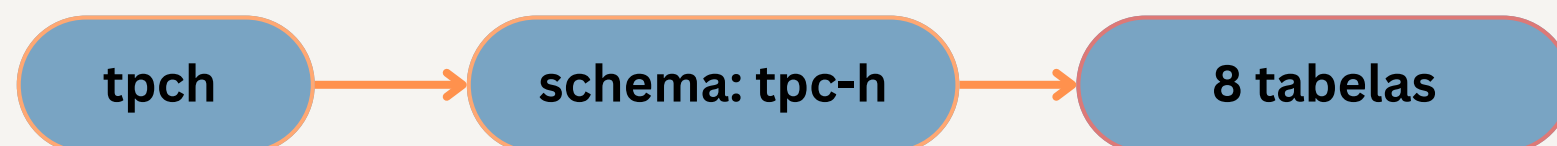
BENCHMARK

Estrutura dos Dados: tabela com as 8 entidades e quantidade de registros

Tabela	Registro
part	20.000
supplier	1.000
partsupp	80.000
customer	15.000
orders	150.000
lineitem	600.572
nation	25
region	5

POSTGRESQL

Criação do Banco de Dados



- Tipos adequados: TIMESTAMP para datas, DECIMAL(12,2) para valores monetários
- Chaves primárias definidas em todas as tabelas
- Chaves estrangeiras adicionadas após a importação

```
CREATE SCHEMA "tpc-h";
SET search_path TO "tpc-h";

CREATE TABLE ORDERS (
    O_ORDERKEY      INTEGER      NOT NULL,
    O_CUSTKEY        INTEGER,
    O_ORDERSTATUS    CHAR(1),
    O_TOTALPRICE     DECIMAL(12,2),
    O_ORDERDATE      TIMESTAMP,
    O_ORDERPRIORITY  CHAR(15),
    O_CLERK          CHAR(15),
    O_SHIPPRIORITY   INTEGER,
    O_COMMENT        TEXT,
    CONSTRAINT ORDERS_PK PRIMARY KEY (O_ORDERKEY)
);
```


POSTGRESQL

Importação dos Dados

- Arquivos .tbl com delimitador |
- Problema: | extra no final de cada linha
- Solução: coluna auxiliar stub
- Comando COPY do PostgreSQL

```
ALTER TABLE orders ADD COLUMN stub CHAR(1);  
COPY orders(..., stub) FROM 'orders.tbl'  
WITH (FORMAT csv, DELIMITER '|', QUOTE E'\x01');  
ALTER TABLE orders DROP COLUMN stub;
```

POSTGRESQL

Script de Benchmark

- 22 queries fornecidas como templates pelo professor
- Parâmetros em branco preenchidos conforme especificação oficial do TPC-H
- Query 15 exigiu tratamento especial: CREATE VIEW → SELECT → DROP VIEW

POSTGRES SQL

Consultas SQL

- Node.js + biblioteca pg
- Lê cada .sql da pasta queries/
- Mede tempo com console.time()
- Exibe amostra de 10 linhas do resultado

MONGODB

CARACTERÍSTICAS

Mapeamento

Como os objetos se
traduzem para o banco

Armazenamento Físico

Tuplas Relacionais estritas

vs.

Documentos BSON

Filosofia de Modelagem

Normalização e

Integridade

vs. **Data Locality e**

Desnormalização

Escalabilidade

Vertical

vs.

Horizontal / Sharding

MONGODB

CONSTRUINDO O BANCO

Armazenamento e “Fusão” dos Dados

- Data Locality
- Batch Processing
- \$push → bulkWrite

Índices

- Estruturas de árvore para busca
- Multikey indexes
- Covered index → query 13

```
bulkOps.push({
  updateOne: {
    filter: { _id: l_orderkey },
    update: { $push: { lineitems: item } }
  }
});

if (bulkOps.length >= 5000) {
  await ordersCol.bulkWrite(bulkOps);
  totalItems += bulkOps.length;
  process.stdout.write(`\rItens aninhados: ${totalItems}`);
  bulkOps = [];
}
```

```
await ordersCol.createIndex({ "lineitems.l_shipdate": 1 });
await ordersCol.createIndex({ "lineitems.l_receiptdate": 1 });
await ordersCol.createIndex({ "lineitems.l_commitdate": 1 });
await ordersCol.createIndex({ "lineitems.l_partkey": 1 });
await ordersCol.createIndex({ "lineitems.l_suppkey": 1 });
await ordersCol.createIndex({ "lineitems.l_returnflag": 1, "lineitems.l_linestatus": 1 });
await ordersCol.createIndex({ "lineitems.l_shipmode": 1 });
```

```
await ordersCol.createIndex({ o_custkey: 1, o_comment: 1 });
```

MONGODB

CONSULTAS PARA O NOSQL

Traduzindo SQL para Aggregation Pipeline

Declarativo vs. Procedural

```
return await ordersCol.aggregate([
  { $unwind: "$lineitems" },
  { $match: { "lineitems.l_shipdate": { $lte: new Date('1998-12-01') } } },
  {
    $group: {
      _id: { returnflag: "$lineitems.l_returnflag", linestatus: "$lineitems.l_linestatus" },
      sum_qty: { $sum: "$lineitems.l_quantity" },
      sum_base_price: { $sum: "$lineitems.l_extendedprice" },
      sum_disc_price: { $sum: { $multiply: ["$lineitems.l_extendedprice", { $subtract: [1, "$lineitems.l_discount"] }] } },
      sum_charge: { $sum: { $multiply: [{ $multiply: ["$lineitems.l_extendedprice", { $subtract: [1, "$lineitems.l_discount"] }] }, { $add: [1, "$lineitems.l_tax"] }] } },
      avg_qty: { $avg: "$lineitems.l_quantity" },
      avg_price: { $avg: "$lineitems.l_extendedprice" },
      avg_disc: { $avg: "$lineitems.l_discount" },
      count_order: { $sum: 1 }
    }
  }
],
```

RESULTADOS

Comparativo de desempenho entre o modelo Relacional e Orientado a Documentos.

- **Objetivo: Analisar os gargalos de cada SGBD executando as 22 consultas analíticas do TPC-H.**

RESULTADOS

Ambiente de Execução

- Testes rodados na mesma máquina física para garantir justiça no comparativo:

- Processador: Intel Core i7 (7500U)
- Memória RAM: 16 GB
- Armazenamento: SSD + HDD
- Isolamento: Variáveis de hardware (leitura de disco e RAM) são os maiores gargalos do TPC-H.

RESULTADOS

O Desafio da Ingestão de Dados

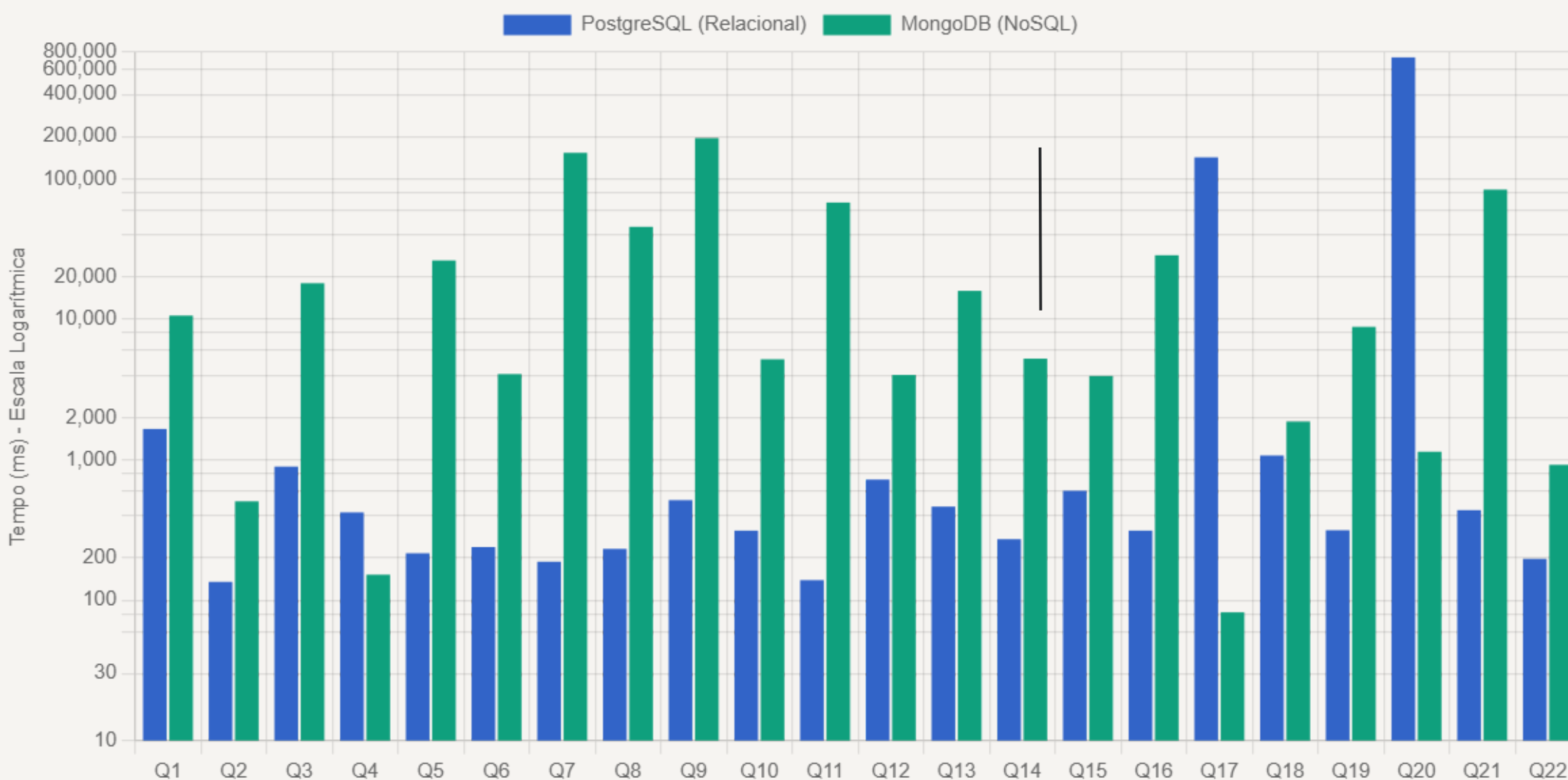
- **Volume massivo:** Tabela de itens (lineitem) contém mais de 600 mil registros.
- **PostgreSQL:** Inserção nativa e rápida (comando COPY).
- **MongoDB:** Inserção complexa (Script Node.js).
- **Leitura sob demanda (streams).**
- **Envio de dados em "pacotes" de 5.000 documentos (BulkWrite) para não estourar a memória.**

RESULTADOS

Estratégias de Modelagem

- **PostgreSQL:**
- **Dados mantidos em tabelas separadas (Clientes, Pedidos e Itens).**
- **Exige múltiplos cruzamentos algébricos pesados (JOINS) na hora da consulta.**
- **MongoDB:**
- **Aninhamento (Embedding): A tabela de itens foi inserida inteira dentro do Pedido.**
- **Multikey Indexes: Criação de índices especiais que apontam direto para o fundo da lista aninhada.**

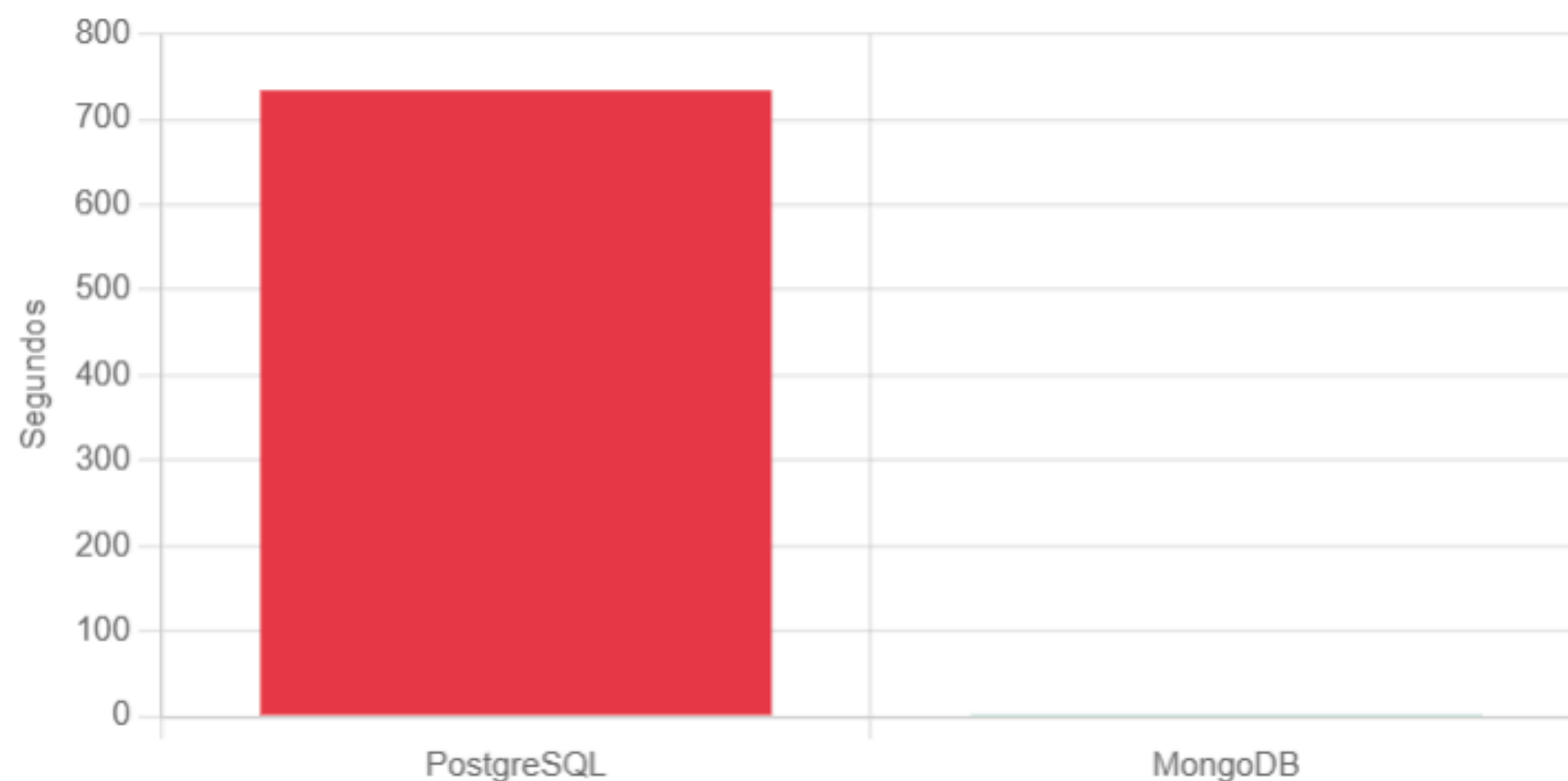
RESULTADOS



- O PostgreSQL venceu 19 de 22 consultas.
- O TPC-H é um teste nativamente relacional (focado em somatórios cruzados).
- Simular JOINS no MongoDB via Aggregation Pipeline é exaustivo: exige desmanchar a lista de itens, calcular e reagrupar (Ex: Q1 levou 10s no MongoDB vs 1.5s no Postgres).

RESULTADOS

O Caso Crítico: Query 20



- O Cenário Absurdo: PostgreSQL = 12 minutos | MongoDB = 1,1 segundo.
- O que houve? O PostgreSQL se perdeu com as subconsultas e traçou a pior rota de busca.
- A Prova: Ao reescrevermos a mesma consulta trocando por JOINS diretos, o tempo no Postgres caiu de 12 minutos para menos de 1 segundo. |

RESULTADOS

Conclusão: A Arquitetura dita a Performance

- **Flexibilidade imprevisível: Se o sistema precisa cruzar dados complexos aleatoriamente, o modelo Relacional continua imbatível.**
- **Hierarquia direta: Se o sistema puxa um dado e seus "filhos" (como um pedido e seus itens), embutir isso no MongoDB transforma a leitura num "fogete".**
- **Todo o nosso código-fonte está no GitHub. Dúvidas?**